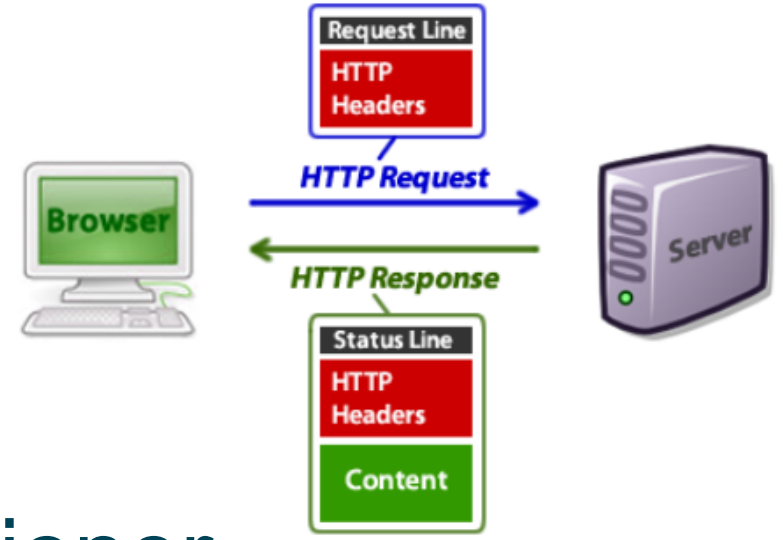
Høgskulen
på Vestlandet



DAT108
Programmering og webapplikasjoner

F17 – Requester og Form-input

Lars-Petter Helland
Bergen
12. oktober 2026

Oppsummert fra sist - 1

Vi skal bruke **Spring MVC** m/ **Thymeleaf** til å lage Java-applikasjoner som kjøres på en webtjener. Spring MVC er et rammeverk som lar oss uttrykke det vi ønsker å få gjort på en grei måte.

Vi skal bruke **Spring Boot** "på toppen av" Spring MVC for enklere prosjektoppsett og konfigurasjon.



Oppsummert fra sist - 2

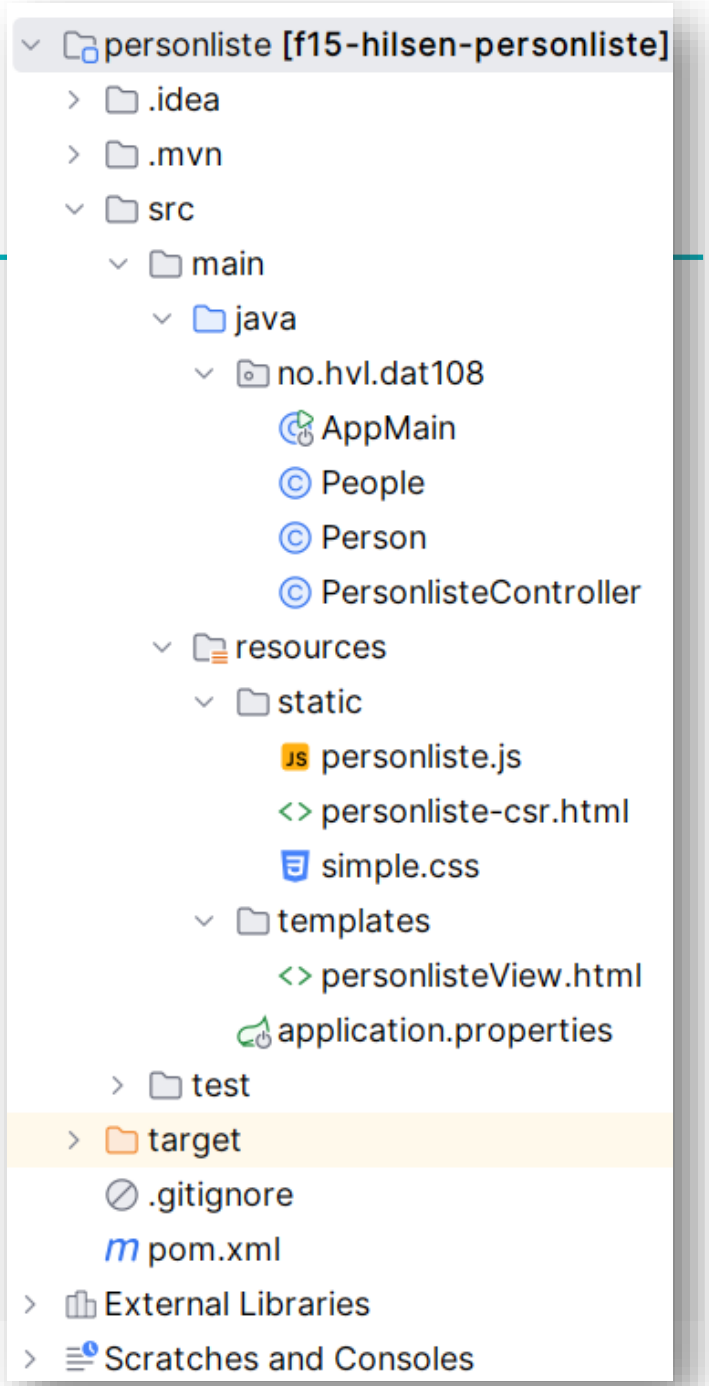
https://maven.apache.org/

Maven™

Prosjektoppsett og konfigurasjon i våre prosjekter skal nå styres av maven.

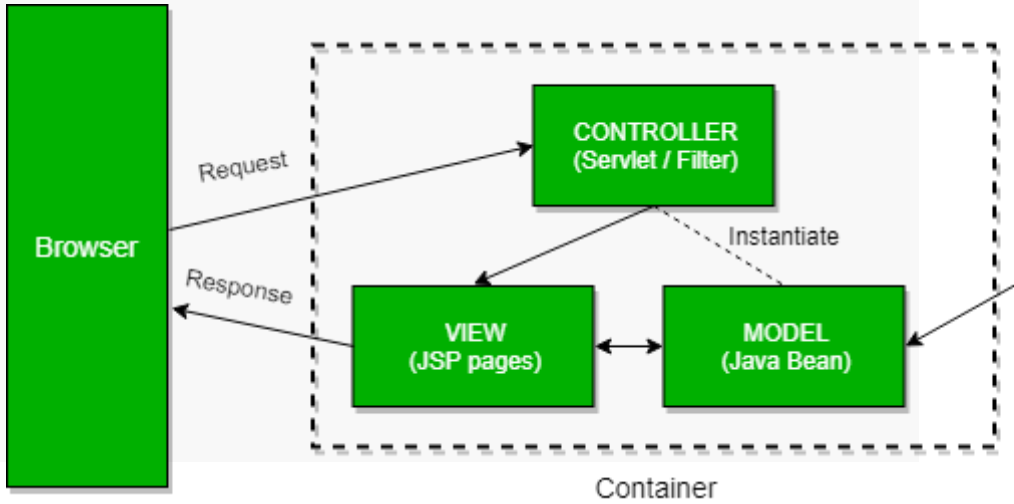
Alt oppsett er beskrevet (direkte eller indirekte) i **pom.xml** i roten av prosjektet.

Prosjektstruktur i maven-prosjekter er standardisert som vist til høyre



Oppsummert fra sist - 3

- Spring MVC-rammeverket baserer seg på en såkalt **Model-View-Controller**-arkitektur.
- Denne innebærer **rollefordeling** og **samarbeid** etter et fast mønster når man skal lage funksjonalitet som svarer på en forespørsel fra en nettleser.
- Den delen som mottar og behandler forespørselen kalles en **controller**. Programmeres i **Java**.
- Et viktig ansvar for en controller er å bygge opp en **model** med data som trengs i responsen. **JavaBean**.
- Hvis responsen f.eks. skal være en HTML-side er det ikke gunstig å lage denne direkte i Java. I stedet:
- Controlleren "sender ballen videre" til et **view** (også en programbit) som genererer HTML-siden ut fra mal + model-dataene. Vi vil bruke **Thymeleaf** som malspråk.



Oppsummert fra sist - 4

- Spring MVC bruker i stor grad annoteringer til å angi hvordan programkode skal forstås.
- Her er en typisk controller:

```
@Controller
public class HalloController {

    @RequestMapping(value = "/hallo")
    public String siHallaien(
            Model model,
            @RequestParam String navn) {

        model.addAttribute("til", navn);
        return "halloView";
    }
}
```

Denne klassen er en Controller

Som svarer på en request til http://.../hallo

Forventer en request-parameter "navn"

Legger inn en model-attributt "til" med verdien i navn

Spiller ballen videre til viewet "halloView"

Oppsummert fra sist - 5

- **Thymeleaf** brukes til å kombinere statisk HTML og dynamiske data for å generere innhold i en respons.
- Thymeleaf (-parseren) blir kjørt på serveren !!
- Returverdien etter kjøring er det ferdige genererte HTML-dokumentet.
- Et par eksempler:

  Å vise en verdien til modell-attributt

  ```
  <p th:text="${melding}" />
  ```
  gir oss `<p>Hei på deg</p>`

  Å iterere over en samling

  ```
  <p th:each="m: ${meldinger}" th:text="${m}" />
  ```
  gir oss flere p-tagger

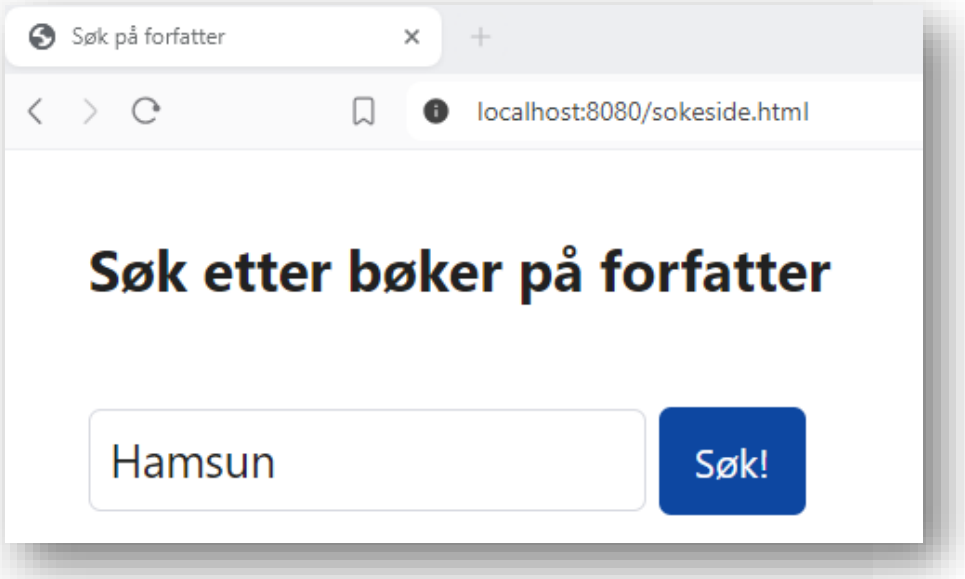
  Å trekke ut objektegenskaper

  ```
  <p th:text="${person.navn}" />
  ```
  gir oss `<p>Arne</p>`

Oppsummert fra sist - 6

- Man lager et skjema i HTML ved å bruke taggene <form> og <input>, f.eks. slik:

```
<form action="noenboker" method="get">
    <input type="text" name="forfatter">
    <input type="submit" value="Søk!">
</form>
```



- Når man "sender inn" skjemaet, enten ved å taste Enter eller trykke på knappen, vil dette resultere i følgende HTTP **GET**-request:

  http://.../**noenboker**?**forfatter**=Hamsun

I dag

- HTTP-meldinger
- GET- og POST-meldinger
- Headere og respons-koder

- HTML-skjema, <form> og <input>
- Når bruke GET og når bruke POST
- Mange ulike typer <input>

- Mapping av request-parametre til Java-variabler, @RequestParam
- Bruke 3.parts bibliotek i et Maven-prosjekt
- Hidden-parametre og sending av indeksverdier

Dagens lesestoff

- Headere og Response-koder: Her holder vi oss til det som er forelest og gitt på øvinger

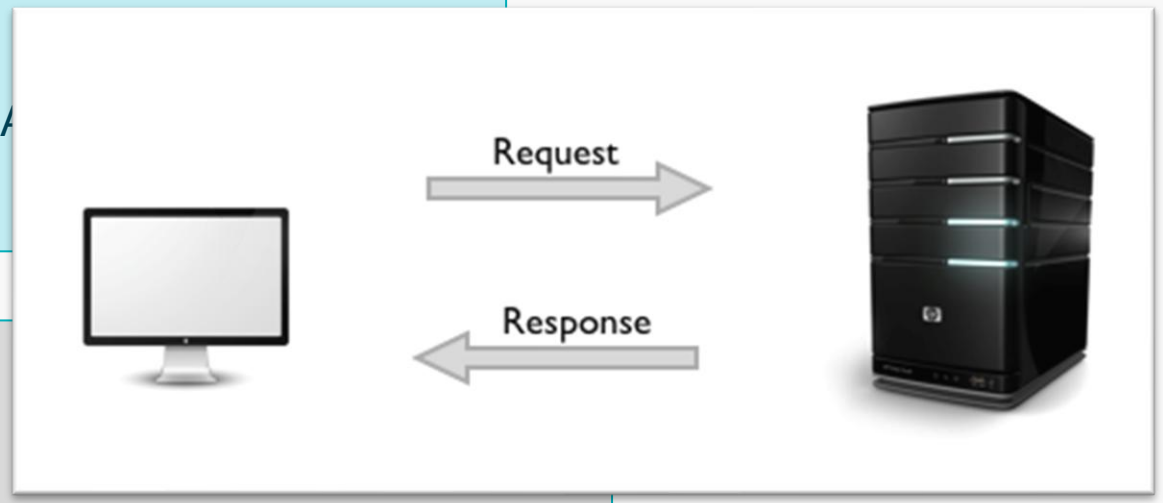
- HTML Forms (artikkel) https://www.w3schools.com/html/html_forms.asp
- HTML Input Types (artikkel) https://daily-dev-tips.com/posts/html-input-types/

- Spring @RequestParam Annotation (artikkel) https://www.baeldung.com/spring-request-param

- View-maler: Thymeleaf CheatSheet for DAT108 (pdf lagt ut i Canvas)

Web = HTTP = Request-Response

```
GET /examples/ HTTP/1.1
Host: localhost:8080
User-Agent: Mozilla/5.0 (Windows NT 5.1; rv:23.0) Gecko/20100101 Firefox/23.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: no,en;q=0.5
Accept-Encoding: gzip, deflate
Cookie: JSESSIONID=62B6246DB2D015DB5526066F71A
Connection: keep-alive
```




```
HTTP/1.1 200 OK
Server: Apache-Coyote/1.1
Accept-Ranges: bytes
Etag: W/"1209-1367736908000"
Last-Modified: Sun, 05 May 2013 06:55:08 GMT
Content-Type: text/html
Content-Length: 1209
Date: Fri, 09 Aug 2013 11:30:57 GMT

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0 Transitional//EN">
<HTML><HEAD><TITLE>Apache Tomcat Examples</TITLE>
...
```

HTTP GET-forespørsel

- GET er den forespørselstypen som er aller mest brukt:
  - Når vi henter en side ved å skrive URL-en
  - Når vi følger en lenke
  - Når vi sender skjemadata (hvis vi ikke ber om POST). Skal se på i dag.
- Info blir da lagt i selve URL-en. Eksempler:
  - http://www.vg.no/
  - http://localhost:8080/f17/Hallo?navn=Per
  - https://www.google.no/search?source=hp&q=h%C3%B8gskolen+p%C3%A5+vestlandet&gs_lcp=CgZwc3ktYWIQAzICCAAyAggAMgIIADICCA ...

GET-forespørsel eksempel

http://localhost:8080/f17/Hallo?navn=Per

```
GET /f17/Hallo?navn=Per HTTP/1.1
Host: localhost:8080
User-Agent: Mozilla/5.0 (Windows NT 5.1; rv:23.0) Gecko/20100101 Firefox/23.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: no,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://localhost:8080/f17/navneskjema.html
Connection: keep-alive
```

HTTP POST-forespørsel

- POST er den forespørselstypen som brukes:
  - Når vi sender skjemadata (hvis vi spesifiserer POST). Skal se på i dag.
- Info blir da ikke lagt i selve URL-en
- POST kan være nyttig om vi skal sende masse data, og må brukes om forespørselen skal føre til endring på serversiden.

- NB! Behandling av en POST-forespørsel har som oftest **IKKE et View**, men i stedet en **Redirect til en kvitteringsside**. (dette er for å hindre dobbeltposting ..)

POST-forespørsel eksempel

http://localhost:8080/f17/Hallo

```
POST /f17/Hallo HTTP/1.1
Host: localhost:8080
User-Agent: Mozilla/5.0 (Windows NT 5.1; rv:23.0) Gecko/20100101
Firefox/23.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: no,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://localhost:8080/f17/navneskjema.html
Connection: keep-alive
Content-Type: application/x-www-form-urlencoded
Content-Length: 8
navn=Per
```

Andre typer forespørsler

- HTTP har en del andre verb enn GET og POST. Disse skal vi se på om et par uker. De er:
  - HEAD
  - PUT
  - PATCH
  - DELETE
  - CONNECT
  - OPTIONS
  - TRACE

HTTP Request message format

- Requestlinje bestående av

  <metode> <request-URI> <HTTP-versjon>, f.eks.

  GET /examples/ HTTP/1.1
- Et antall request-headere
- En tom linje
- Evt. en body (hvis metoden er **POST**)

Noen vanlige request-headere

- **accept**: Hvilke type dokument foretrekkes, f.eks. "text/html"
- **accept-charset**: Hvilke tegnsett foretrekkes, f.eks. "UTF-8"
- **accept-language**: Hvilke språk foretrekkes, f.eks. "no,en;q=0.5"
- **authorization**: Brukes i fbm. med tilgangskontroll
- **cookie**: Data lagret i nettleser for dette nettstedet eller denne "sesjonen"
- **refererer**: Hvilken side "kommer man fra"
- **user-agent**: Hvilken nettleser-motor er det
- (headere for caching ...)

Få tak i request-headere i controlleren

- https://www.baeldung.com/spring-rest-http-headers

- Hente verdi fra en header i **Spring MVC**, f.eks.

```
@RequestHeader(HttpHeaders.ACCEPT_LANGUAGE) String spraak
```

- eller alle

```
@RequestHeader Map<String, String> headers
```

HTTP Response message format

- Statuslinje, f.eks.
  HTTP/1.1 200 OK
- Et antall respons-headere. Eneste obligatoriske er:
  Content-Type: text/html (eller noe annet)
- En tom linje
- Evt. en body

HTTP Respons statuskoder

Vi skal ikke se på alle ca. 40 kodene, men nevner de mest vanlige:

100-101 Midlertidig respons. Ikke ferdig.

200-206 Alt gikk greit.

- **200 OK – Forespørselen var vellykket, ...**
- **201 Created - Forespørselen var vellykket, og en ny ressurs har blitt opprettet**

300-307 Omdirigering.

- **302 Found – Siden du skal til er på en annen adresse** (Gitt i Location-header)
- **304 Not Modified – Ressursen du ber om er ikke oppdatert siden sist** (muliggjør caching)

400-417 Feil som skyldes klienten.

- **401 Unauthorized – Du har ikke tilgang til siden / ressursen**
- **404 Not Found – Serveren finner ikke det du leter etter**

500-505 Feil som skylders tjeneren

- **500 Internal Server Error – Noe gikk feil på serveren**

Noen vanlige Respons-headere

- Respons-headere er en viktig del av responsen. Det er i hovedsak webtjeneren som tar seg av å sette respons-headere.
- Ramser ikke opp nå. Tar et par, og resten etter hvert som de blir aktuelle.

Content-Type

- Dette er den eneste obligatoriske respons-headeren. Den angir MIME-typen til dokumentet som sendes tilbake. Vanlige verdier er **text/html**, **text/plain** og **application/json**

Set-Cookie

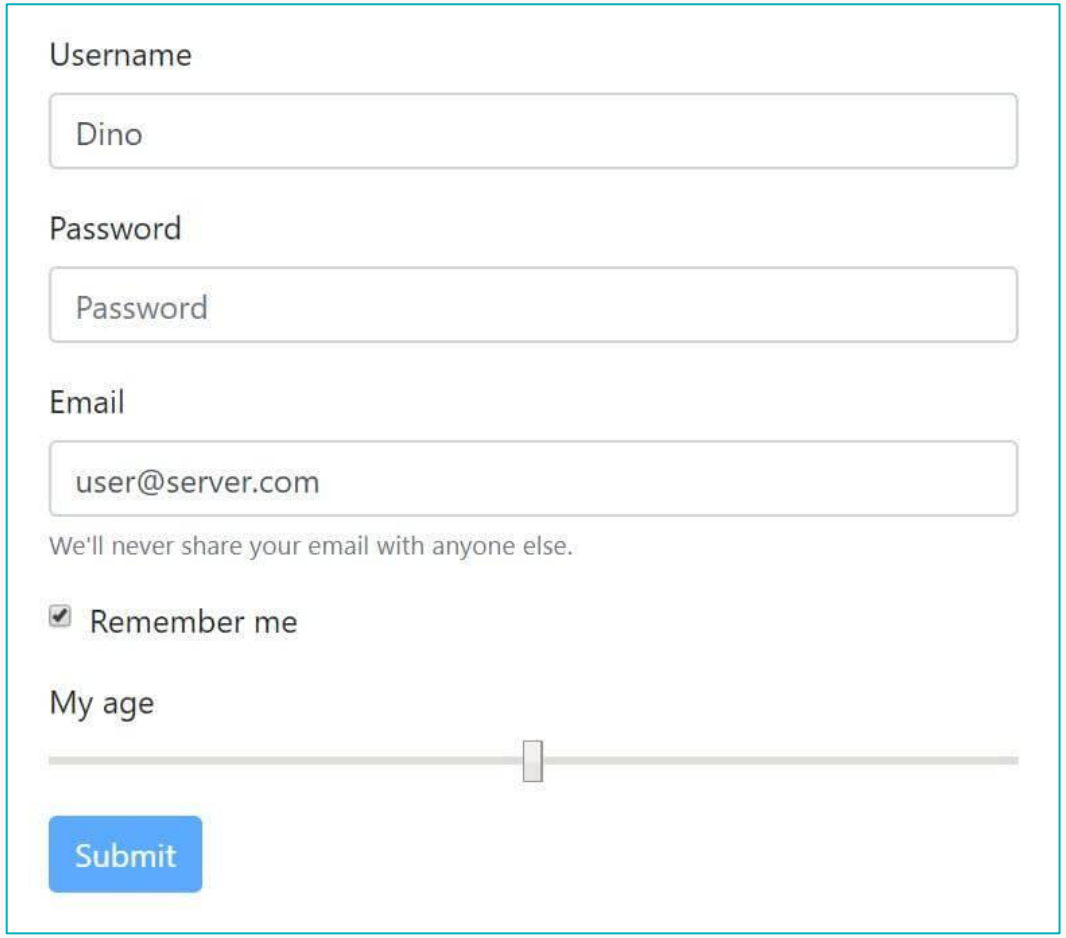
- For å sette en cookie i nettleseren. En cookie er en tekststreng som lagres i nettleseren og er knyttet til dette nettstedet. Nå klienten senere går til nettstedet blir cookien sendt med i forespørselen. Vi skal se på et eksempel neste uke.

- Location
  - Brukes f.eks. i forbindelse med responsekode 302 Found, som er en instruksjon til nettleseren om å gå til adressen som er oppgitt i location. Brukes f.eks. ved en redirect etter en POST-request.

Nå

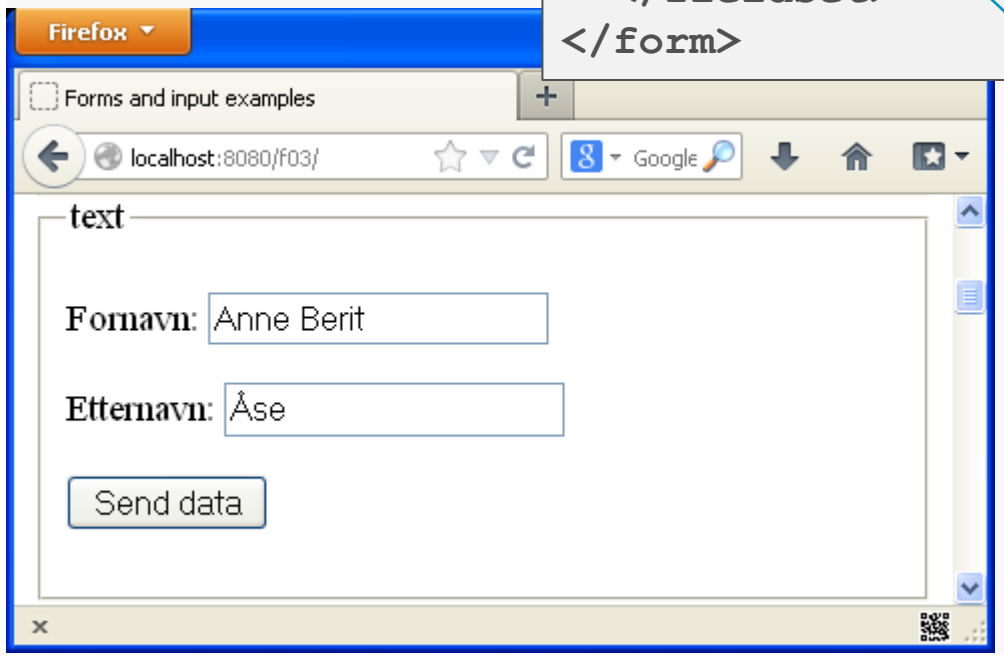

HTML Forms
& Input Types
The Backbone of
User Interaction on
the Web

HTML <form>-tag (på godt norsk: skjema)



HTML form – Eksempel m/ GET

```
<form action="paamelding" method="get">
  <fieldset>
    <legend>text</legend>
    <p>Fornavn: <input type="text" name="fornavn" /></p>
    <p>Etternavn: <input type="text" name="etternavn" /></p>
    <p><input type="submit" value="Send data" /></p>
  </fieldset>
</form>
```



NB! <fieldset> og <legend> er kun **pynt**, og har ingen semantisk betydning

Hva sendes til tjener?

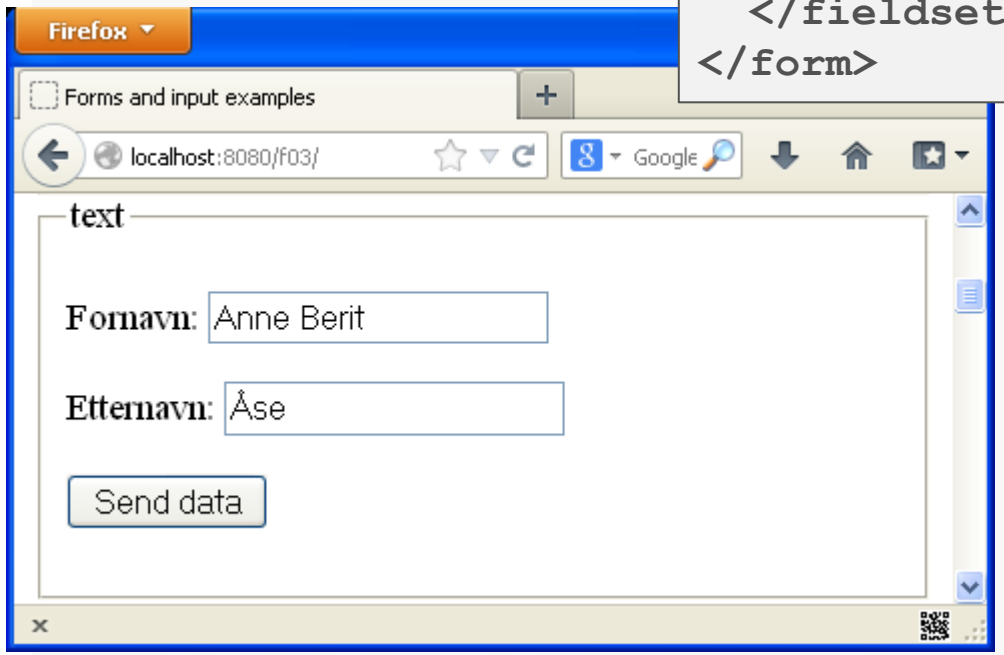
```
GET /f17/paamelding?fornavn=Anne+Berit&etternavn=%C5se HTTP/1.1
Host: localhost:8080
User-Agent: Mozilla/5.0 (Windows NT 5.1; rv:23.0) Gecko/20100101 Firefox/23.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: no,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://localhost:8080/f17/
Connection: keep-alive
```

- Hvilken HTTP-metode benyttes?
- Hvor havner dataene vi tastet inn?
- Hvordan er dataene kodet?

Samme eksempel, men med POST

.

```
<form action="paamelding " method="post">
  <fieldset>
    <legend>text</legend>
    <p>Fornavn: <input type="text" name="fornavn" /></p>
    <p>Etternavn: <input type="text" name="etternavn" /></p>
    <p><input type="submit" value="Send data" /></p>
  </fieldset>
</form>
```



Hva sendes til tjener nå?

```
POST /f17/paamelding HTTP/1.1
Host: localhost:8080
User-Agent: Mozilla/5.0 (Windows NT 5.1; rv:23.0) Gecko/20100101 Firefox/23.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: no,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://localhost:8080/f17/
Connection: keep-alive
Content-Type: application/x-www-form-urlencoded
Content-Length: 34
   fornavn=Anne+Berit&etternavn=%C5se
```

- Hvor havner dataene vi tastet inn nå?
- Hvordan er dataene kodet?
- Hva er Content-Type for noe?

Oppsummert

Fra http://www.w3schools.com/html/html_forms.asp:

HTML forms are used to pass data to a server.

An HTML form can contain input elements like text fields, checkboxes, radio-buttons, submit buttons and more. A form can also contain select lists, textarea, fieldset, legend, and label elements.

- Et **<form>**-element er altså et HTML-element som plasseres "rundt" et eller flere **<input>**-elementer.
- Når formen "**submit**tes" sendes **alle** data i formen til angitt URL.
- Ting vi oppgir i en <form>:
  - **action**="..." angir URL-en vi går til. Er som oftest relativ. Default er ""
  - **method**="..." angir om vi ønsker GET eller POST. Default er "get"

GET eller POST?

- **GET** brukes vanligvis når man skal "hente" informasjon
  - GET kan bokmerkes (lenker kan deles)
  - GET gjør at vi kan skrive parametre direkte i URL
  - Ikke bruk GET om det gir "bivirkninger" på tjeneren

- **POST** brukes vanligvis når man skal "sende" informasjon
  - POST "skjuler" parametre
  - POST kan sende større mengder data
  - Nettlesere gir advarsel mot "dobbeltposting", noe som gjør POST egnet om det gir "bivirkninger" på tjeneren. Mer om dette senere.



Hva gjør vi på tjenersiden?

OK, så vi mottar et "brev" av denne typen:

```
POST /f17/paamelding HTTP/1.1
Host: localhost:8080
User-Agent: Mozilla/5.0 (Windows NT 5.1; rv:23.0) Gecko/20100101 Firefox/23.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: no,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://localhost:8080/f17/
Connection: keep-alive
Content-Type: application/x-www-form-urlencoded
Content-Length: 34
   fornavn=Anne+Berit&etternavn=%C5se
```

Hva gjør vi på tjenersiden?

- I en Java Web-applikasjon bli dette "brevet" automatisk dekodet og representert i et **HttpServletRequest**-objekt.
- Spring-rammeverket vil videre hjelpe oss å hente ut informasjon fra HttpServletRequesten, f.eks. request-parametre og -headere og binde dette til variabler i Controllerne våre.
- Vi kan se på et eksempel.

Eksempel F17a

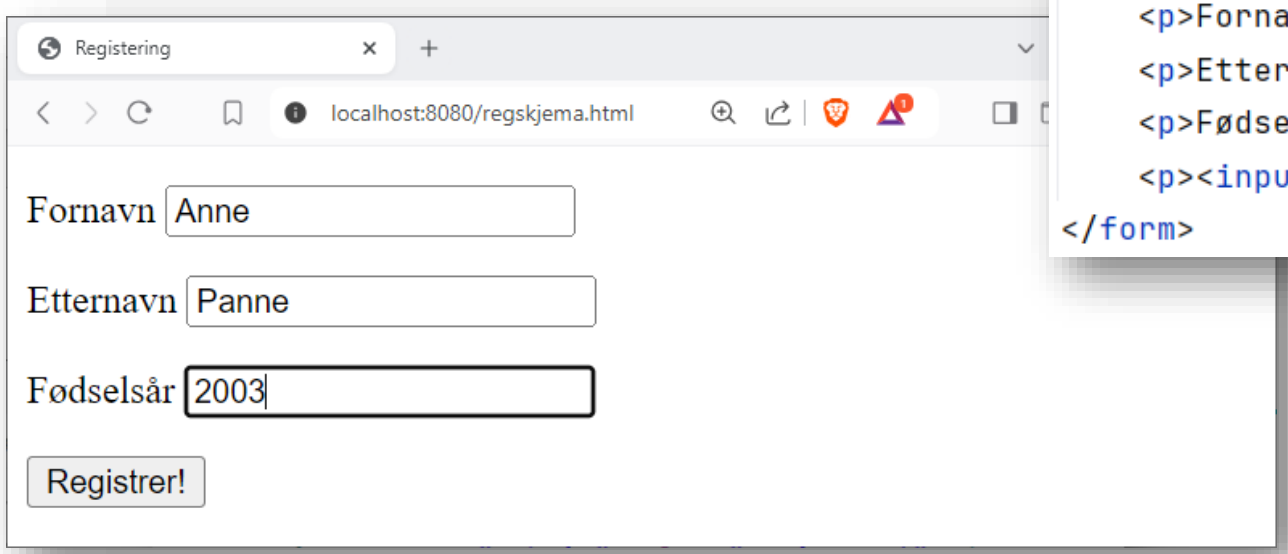
Prosjekt: 📁 f17a-person-regskjema

Skjema-URL: http://localhost:8080/regskjema.html

Controller-URL: http://localhost:8080/registrer



Eksempel F17a - Oppsummert



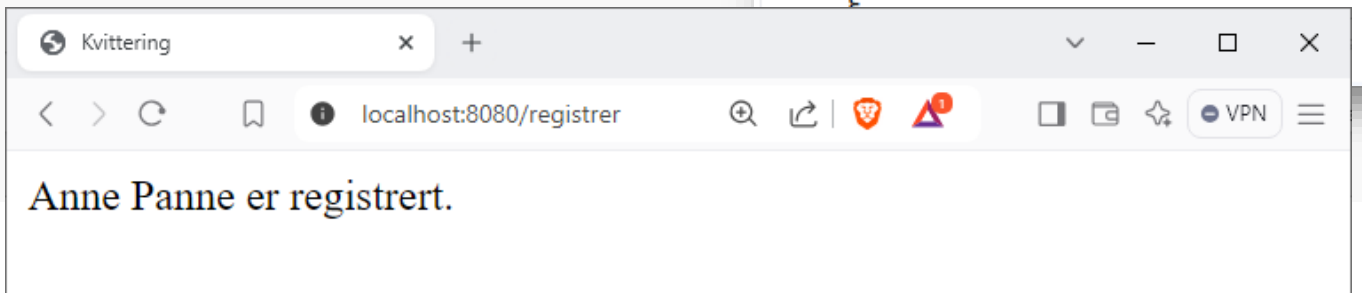

```
<form action="registrer" method="post">
    <p>Fornavn <input type="text" name="fornavn"></p>
    <p>Etternavn <input type="text" name="etternavn"></p>
    <p>Fødselsår <input type="text" name="aar"></p>
    <p><input type="submit" value="Registrer!"></p>
</form>
```

```
@Controller
public class RegistreringController {

    @PostMapping("/registrer")
    public String taImotRegistrering(Model model,
            String fornavn, String etternavn, int aar) {

        Person p = new Person(fornavn, etternavn, aar);
        model.addAttribute("personen", p);
        return "kvitteringView";
    }
```

```
<p th:text="|${personen.fornavn} ${personen.etternavn} er registrert.|"/>
```



Dere skal kjenne til og kunne bruke “alle” HTML input-typer

Det er 22 input-typer i HTML-standarden. En grei oversikt kan dere finne f.eks. her

- https://www.w3schools.com/html/html_form_input_types.asp eller her
- https://daily-dev-tips.com/posts/html-input-types/ eller her
- https://developer.mozilla.org/en-US/docs/Web/HTML/Element/input

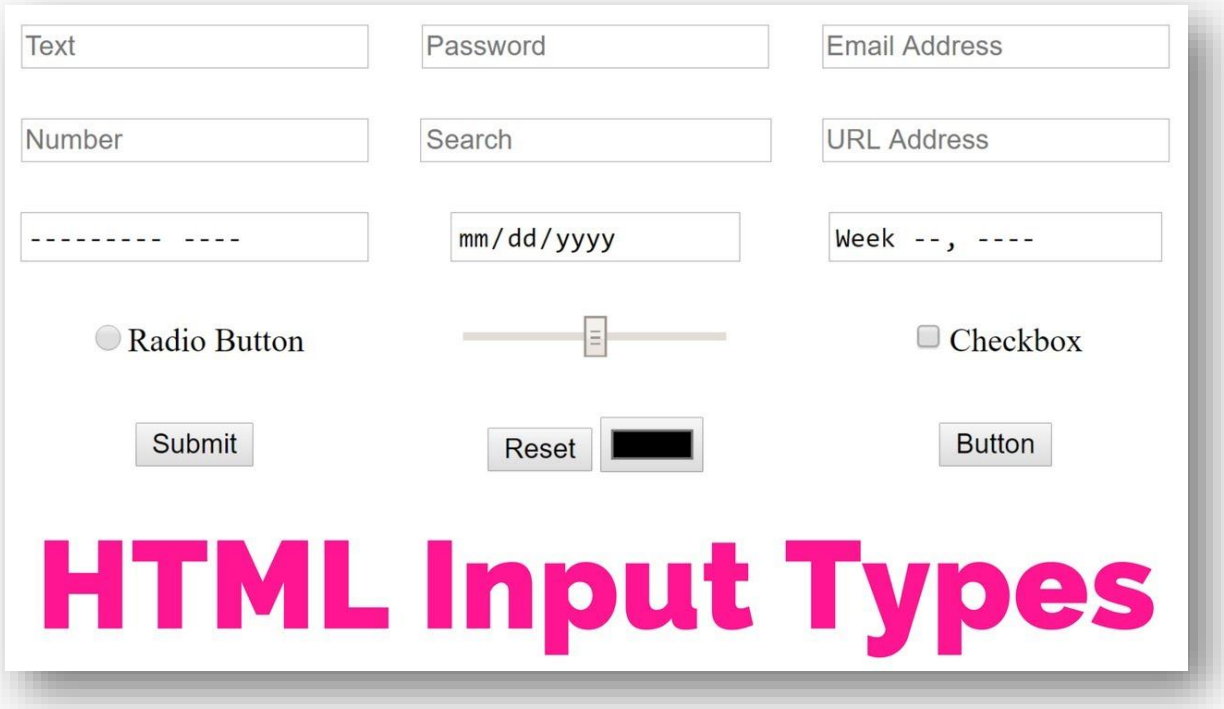
Noen vanlige Input-typer (<input type="..."> og form-elementer

- **text**
- **password**
- email, **number**, search, url, tel, color, range, file
- datetime-local, **date**, month, week, time
- **<textarea ..>**

- **radio**
- **checkbox**
- <select ..>

- **submit**, reset, image
- **<button ..>**

- **hidden**



<form><input>...

Hva skjer nå vi «submitter» formen?

Jo, all input i form-taggen blir sendt som request-parametre, der:

- parameter**navn** blir input-ens name <input name= "..." >
- parameter**verdi** blir det brukeren har gitt inn

Spesialtilfeller / unntak:

- <input type= "checkbox"> sender **flere** parametre med samme navn men **ulik verdi**
- <input type= "hidden"> sender den verdien du har angitt i **value**

Eksempel F17b

Prosjekt: 📁 f17b-boksok-m-sortering

(fortsette litt på eksempel fra sist gang)

- Sortere (radioknapper)
- Velge genre (checkbox-er)

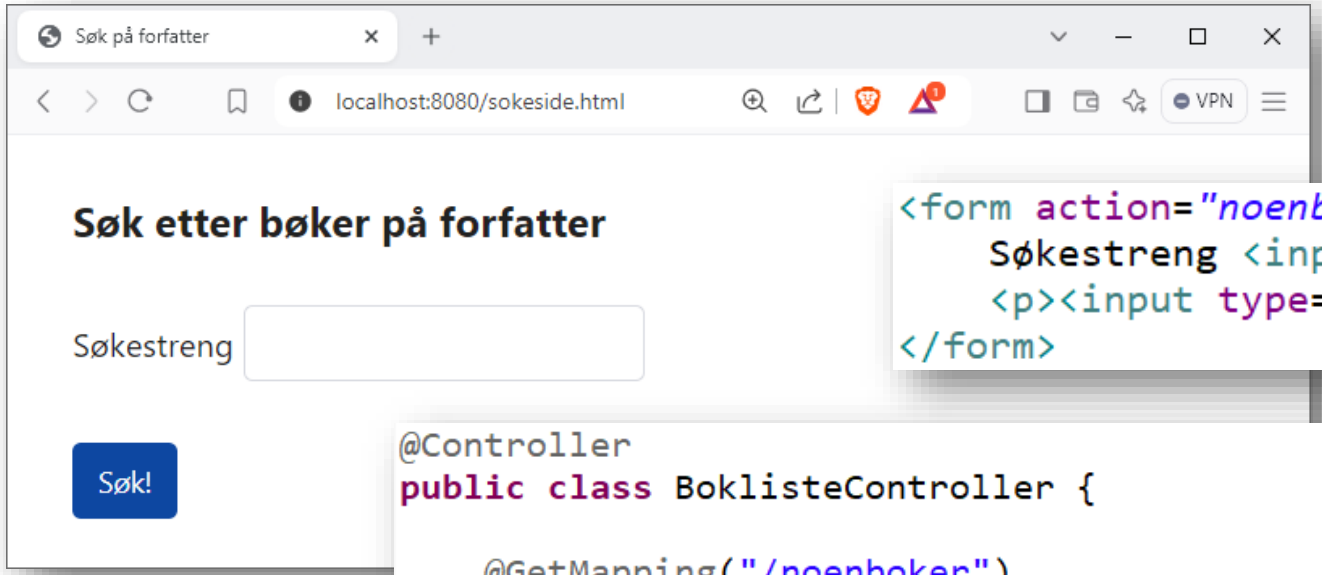


Skjema-URL: http://localhost:8080/sokeside.html

Controller-URL: http://localhost:8080/noenboker

Eksempel F17b - Oppsummert

Utgangspunktet fra sist




```
<form action="noenboker" method="get">
    Søkestreng <input type="text" name="forfatter"><br>
    <p><input type="submit" value="Søk!"></p>
</form>
```

```
@Controller
public class BoklisteController {

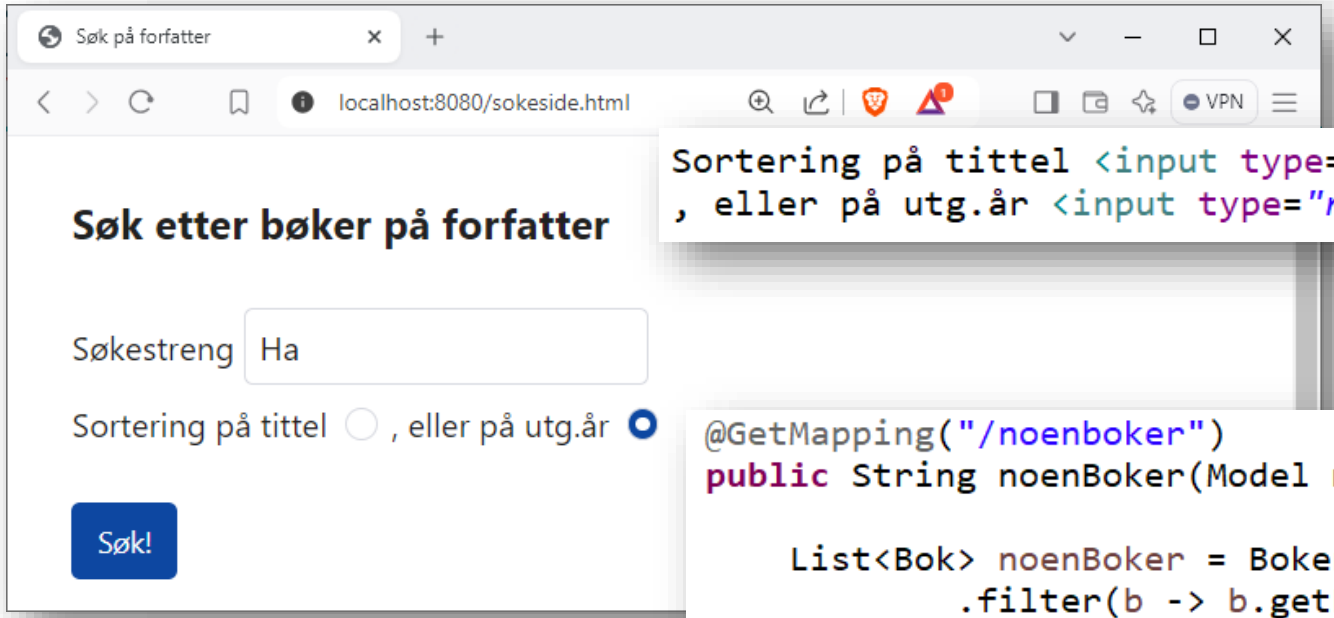
    @GetMapping("/noenboker")
    public String noenBoker(Model model, String forfatter) {

        List<Bok> noenBoker = Boker.alleBoker.stream()
                .filter(b -> b.getForfatter().contains(forfatter))
                .toList();

        model.addAttribute("boker", noenBoker);
        return"boklisteView";
```

Eksempel F17b - Oppsummert

Sortering - Radioknapper - Basic Controller




```
Sortering på tittel <input type="radio" name="sortering" value="paaTittel" checked>
, eller på utg.år <input type="radio" name="sortering" value="paaAar">
```

```
@GetMapping("/noenboker")
public String noenBoker(Model model, String forfatter, String sortering) {

    List<Bok> noenBoker = Boker.alleBoker.stream()
            .filter(b -> b.getForfatter().contains(forfatter))
            .toList();

    if (sortering.equals("paaTittel")) {
        noenBoker.sort((b1, b2) -> b1.getTittel().compareTo(b2.getTittel()));
    } else { // sortering.equals("paaAar")
        noenBoker.sort((b1, b2) -> b1.getUtgivelsesaar() - b2.getUtgivelsesaar());
    }

    model.addAttribute("boker", noenBoker);
    return"boklisteView";
```

Eksempel F17b - Oppsummert

Sortering - Radioknapper - Litt mer avansert kode i Controller (i retning slik jeg ville gjort selv)

```
@Controller
public class BoklisteController {

    @GetMapping("/noenboker")
    public String noenBoker(Model model, String forfatter, String sortering) {

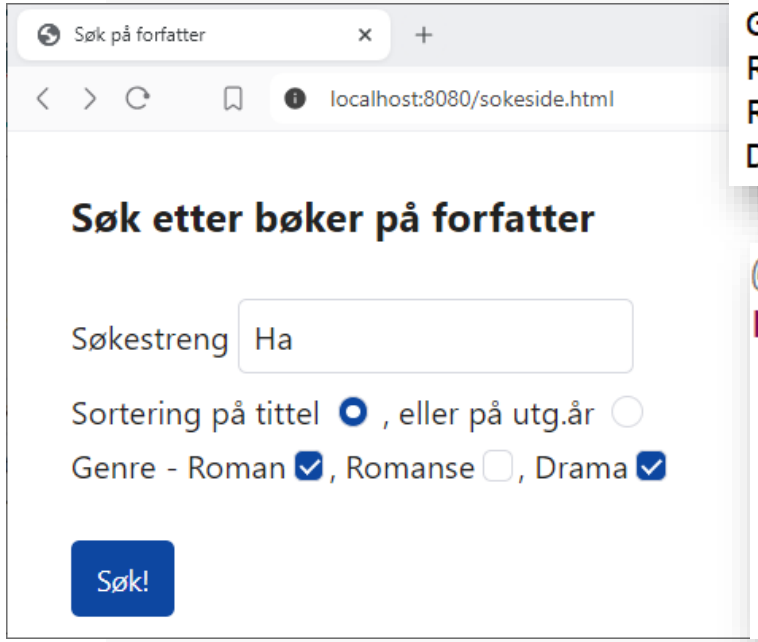
        /* Legger inn aktuelle comparatorer i et Map. Slipper da if-else
         * for å velge comparator, men kan bruke et direkte oppslag i stedet.
         * Dette map-et kunne gjerne vært plassert i Bok-/Boker-klassen i
         * stedet for her i Controlleren.
         */
        Map<String, Comparator<Bok>> comp = new HashMap<>();
        comp.put("paaTittel", Comparator.comparing(Bok::getTittel));
        comp.put("paaAar", Comparator.comparing(Bok::getUtgivelsesaar));

        List<Bok> noenBoker = Boker.alleBoker.stream()
                .filter(b -> b.getForfatter().contains(forfatter))
                .sorted(comp.get(sortering))
                .toList();

        model.addAttribute("boker", noenBoker);
        return"boklisteView";
```

Eksempel F17b - Oppsummert

Filtrering på genre




```
Genre -
Roman<input type="checkbox" name="genre" value="Roman" checked>,
Romanse<input type="checkbox" name="genre" value="Romanse">,
Drama<input type="checkbox" name="genre" value="Drama"><br>
```

```
@Controller
public class BoklisteController {

    @GetMapping("/noenboker")
    public String noenBoker(Model model, String forfatter,
            String sortering, String[] genre) {

        List<String> valgteGenre = List.of(genre);
```

```
List<Bok> noenBoker = Boker.alleBoker.stream()
        .filter(b -> b.getForfatter().contains(forfatter))
        .filter(b -> valgteGenre.contains(b.getGenre()))
        .sorted(comp.get(sortering))
        .toList();
```

```
<input type="hidden" name="myId" value="1234">
```

Gir et «skjult» felt.

NB! Dette ikke er en sikkerhetsmekanisme, men et verktøy for å sende "bakgrunnsdata" som brukeren slipper å se eller skrive inn manuelt.

Bruksområder:

1. Id-er (f.eks. artikkel-id) som trengs, men som brukeren ikke trenger å se
2. Token for å hindre Cross-Site Request Forgery (CSRF)
3. Tidligere brukerinput for skjemaer som går over flere sider/steg
4. Sporing av kontekst eller kilde (analyse / markedsføring)

Aldri legg sensitive ting som produktpriser eller brukertilganger i slike felt.

Eksempel F17c

Demo av bruk av indeks-data og hidden-parameter



Prosjekt: 📂 f17c-hidden-kontoer

Skjema-URL: http://localhost:8080/kontovalg.html

Controller-URL: http://localhost:8080/kontovalg

Eksempel F17c - Oppsummert

1. Hvis vi ikke ønkser å sende data (f.eks. kontonumre) over nettet, kan vi i stedet sende indekser til bruk i oppslag på serveren

```
<p>Velg konto:</p>
<!-- Bruk av indekser for oppslag på server -->
<input type="radio" name="kontovalg" value="0" checked>Brukskonto<br>
<input type="radio" name="kontovalg" value="1">Sparekonto <br>
<input type="radio" name="kontovalg" value="2">Aksjekonto <br>
```

2. Hvis vi trenger/ønsker å sende ekstra data (som brukeren ikke taster inn eller ser), kan vi bruke en hidden input-type.

```
<input type="hidden" name="usersecret" value="7FQ12lkj324kj23h4k23j5657k">
```

Videre

Neste gang(er) :

- Validering av brukerinput
- ...

...

Det er laget en frivillig oppgave (om norske konger) dere kan jobbe med på labben. ..
Dere kan også dikte opp deres egen oppgave som har en **html-side** med en **<form>** og noe **<input>**, som gjør en request til en **controller**, og der det lages en dynamisk respons med **thymeleaf**.

Oblig4 legges ut ila et par dager. Det er kort frist, så det er bare å sette i gang.